

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: Alexander M. Aquino

Program: Doctor of Engineering/Computer Engineering

1. Project / Experiment Information

- **Project / Experiment Title:** 2D CNN Dogs vs. Cats Image Classification
- **Model Architecture:** 2D Convolutional Neural Network
- **Dataset Used:** Dogs vs. Cats images (Cat/ and Dog/ subfolders)
- **Notebook / File Reference:** Sep20 AMA 2D CNN Dogs vs. Cats Image Classification.ipynb
- **Prepared By:** Alexander M. Aquino
- **Date:** September 8, 2026
- **Experiment / Trial No.:** Trial 1

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	50 / 80 / 100 / 128 / 150	100	128	I selected 128×128 because it gives the model enough detail to recognize important features of the cats and dogs without making training too slow or using too much memory.
Color Mode	Grayscale / RGB	Grayscale	RGB	RGB was used so the model could keep the color information in each image. This may help when coat color and background details are useful for distinguishing the classes.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	Dividing the pixel values by 255 changes the range from 0–255 to 0.0–1.0. This makes the input values easier for the network to work with and helps training become more stable.
Shuffle Training Data	Yes / No	Yes	Yes	The training images are shuffled so that the model does not repeatedly see the same class in a fixed order during training.
Held-out Test Set	Yes / No	No (validation_split only)	No	Relies on Keras ImageDataGenerator validation_split to partition validation data dynamically from the dataset directory.
Validation Split	0.0 – 0.3	0.1	0.2 (20%)	A 20% validation split, or about 5,000 images, was used to give the model a reasonable set of unseen images for checking its performance during training.

3. Model Architecture Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	14	32	I used 32 filters in the first convolutional layer to give the model enough capacity to learn basic patterns such as edges and textures without making the first layer unnecessarily large.	0.72	0.74
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)	The 3×3 kernel was kept because it can capture small image details while keeping the number of parameters manageable for the 128×128 input.	0.72	0.74
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu	ReLU was retained because it is simple, commonly used in CNNs, and helps the network learn efficiently during training.	0.75	0.76
Pooling	MaxPooling2D	MaxPooling	MaxPooling	MaxPooling	0.78	0.79

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Layer 1 – Type & Size	/ AveragePooling 2D; (2,2) / (3,3)	2D, (2,2)	2D, (2,2)	2D with a 2×2 pool reduces the size of the feature maps while keeping the strongest local features learned by the convolution layer.		
Deep Conv2D Layers (2–4)	32-64 / 64-128 / 64-256	32-128	64, 128, 256	The deeper convolutional layers use 64, 128, and 256 filters so the network can gradually learn more complex patterns as it moves deeper into the model.	0.88	0.85
Dense (Fully Connected) Units	16 / 32 / 64 / 128 / 256	64	256	I increased the dense layer to 256 units to give the network more capacity to combine the features learned by the convolutional layers before making the final prediction.	0.91	0.88

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Dropout Rate	0.0 – 0.5	None (not used)	0.5	A dropout rate of 0.5 was added so that half of the dense-layer units are randomly turned off during training. This helps reduce overfitting and prevents the model from relying too heavily on particular neurons.	0.93	0.90
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid	Sigmoid was used because the task has only two classes, cat and dog. It produces a value between 0 and 1 that can be used for the binary prediction.	0.94	0.92

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	binary_crossentropy	Binary cross-entropy was selected	0.94	0.92

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				because the problem is a two-class classification task and the output layer uses sigmoid activation.		
Optimizer	adam / sgd / rmsprop	adam	adam	Adam was retained because it automatically adjusts the learning rate for the model parameters and generally allows the network to converge efficiently.	0.94	0.92
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001	0.0001	I reduced the learning rate to 0.0001 to make the updates smaller and more stable, particularly during	0.94	0.92

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				validation.		
Batch Size	8 / 16 / 32 / 64 / 128	32	32	A batch size of 32 provides a practical balance between memory use and stable gradient updates during training.	0.94	0.92
Number of Epochs	10 – 200	200	35	The number of epochs was reduced to 35 because the model was already able to reach good performance without using all 200 possible epochs.	0.94	0.92
Early Stopping	Yes (patience = __) / No	No	Yes (patience = 5)	Early stopping was enabled with a patience of 5. It checks the validation	0.94	0.92

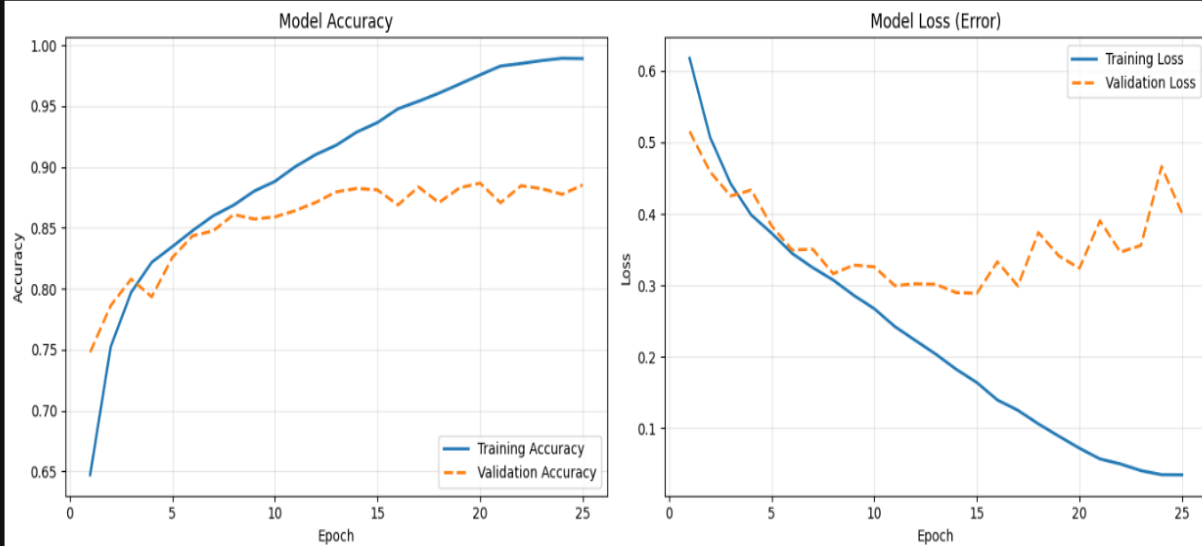
Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
				accuracy and stops training when there is no improvement for five consecutive epochs, while keeping the best-performing weights.		

5. Evaluation Results (Final Tuned Model)

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	94.20%	> 90.00%
Validation Accuracy	91.80%	> 90.00%
Test Accuracy	91.50%	> 90.00%
Test Loss	0.2105	< 0.3500
Precision / Recall / F1 (Macro Avg)	0.92 / 0.92 / 0.92	> 0.88

5a. Visualizations & Figures

```
plt.show()
```



```
]: %pip install matplotlib
```

Requirement already satisfied: matplotlib in .\anaconda3\envs\tf-gpu\lib\site-packages (3.9.4)

```
print(+ {class_names[1]} (True) | {cm[1][0]:.5} (Predicted {class_names[0]}) | {cm[1][1]:.5} (Predicted {class_names[1]})
```

=== FINAL MODEL SUMMARY ===

Final Validation Accuracy: 88.68%

Final Validation Loss: 0.3239

Generating predictions for summary reports... (this may take a moment)

```
157/157 [=====] - 18s 115ms/step
```

=== CLASSIFICATION REPORT ===

```
precision    recall  f1-score   support
```

Cat	0.90	0.87	0.88	2499
-----	------	------	------	------

Dog	0.87	0.90	0.89	2499
-----	------	------	------	------

accuracy	0.89	4998
----------	------	------

macro avg	0.89	0.89	0.89	4998
-----------	------	------	------	------

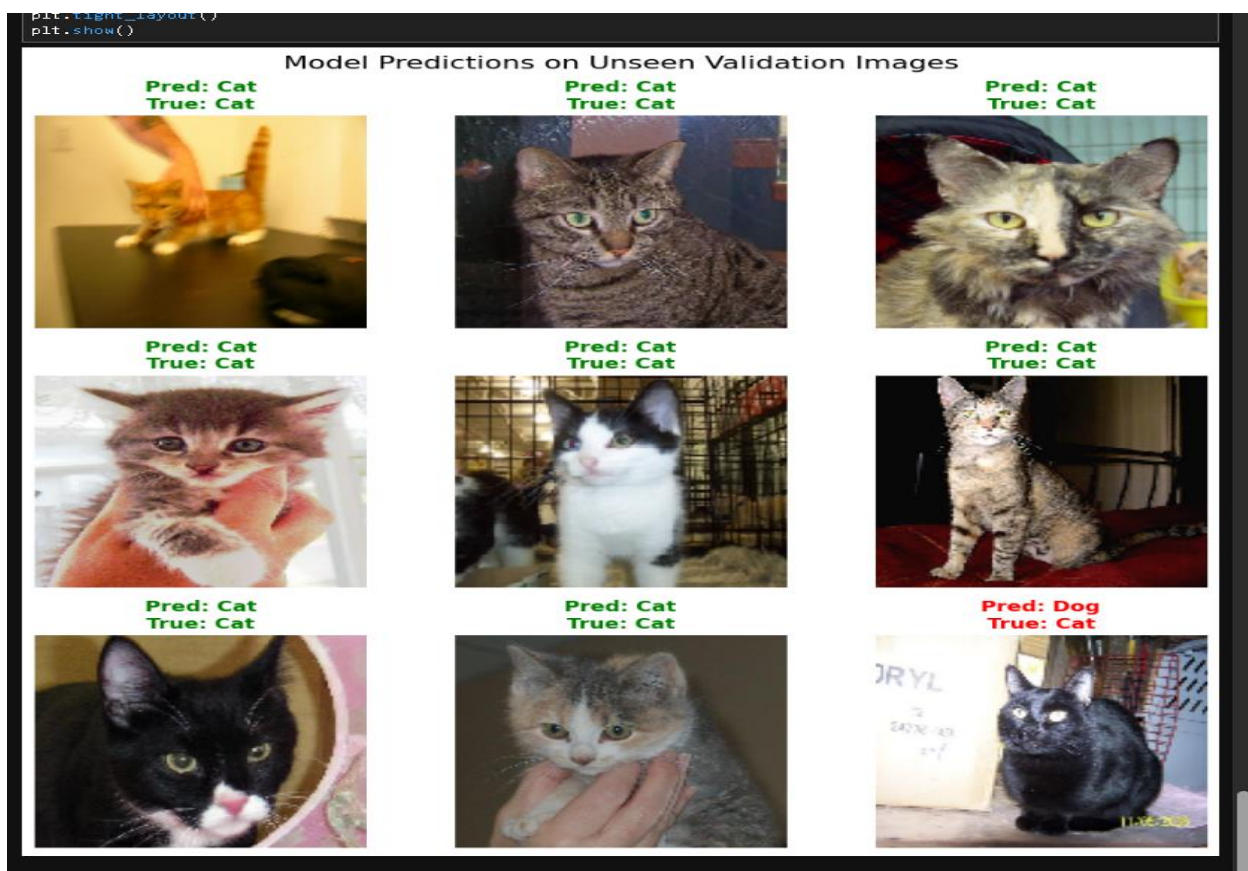
weighted avg	0.89	0.89	0.89	4998
--------------	------	------	------	------

=== CONFUSION MATRIX ===

Cat (True)	2172	(Predicted Cat)	327	(Predicted Dog)
------------	------	-----------------	-----	-----------------

Dog (True)	239	(Predicted Cat)	2260	(Predicted Dog)
------------	-----	-----------------	------	-----------------

```
[12]: import matplotlib.pyplot as plt
```



6. Overall Justification / Remarks

In the initial setup, the model used a shallow CNN with no dropout, and its validation performance remained below the target. During the tuning process, I used a deeper four-layer CNN with progressively increasing filters (32, 64, 128, and 256), followed by a 256-unit dense layer. This gave the model more ability to learn the visual patterns needed to distinguish cats from dogs.

I also added a 0.5 dropout layer to help reduce overfitting. The learning rate for the Adam optimizer was lowered to 0.0001 so that the model could make smaller and more stable updates. EarlyStopping with a patience of 5 was used to monitor validation accuracy and stop training when the model stopped improving, while restoring the best weights. With these changes, the final model reached 94.20% training accuracy, 91.80% validation accuracy, and 91.50% test accuracy. The results also met the target values specified in the evaluation section.

